

CONCEPTO CENTRAL

Busca un valor dentro del arreglo y, cuando lo encuentra, desplaza los elementos posteriores una posición hacia la izquierda para cubrir el espacio. Finalmente, disminuye el tope en 1.

¿CÓMO FUNCIONA?

- Recorrer el arreglo desde la primera posición.
- Comparar cada elemento con X.
- Cuando aparece X, desplazar los elementos de la derecha hacia la izquierda.
- Reducir el valor de tope porque ahora hay un elemento válido menos.

PASO A PASO PARA EXPONER

- 1 Recorrer el arreglo desde la primera posición.
- 2 Comparar cada elemento con X.
- 3 Cuando aparece X, desplazar los elementos de la derecha hacia la izquierda.
- 4 Reducir el valor de tope porque ahora hay un elemento válido menos.

CÓDIGO / LÓGICA JAVA

```
for (int i = 0; i < tope; i++) {  
    if (numerito[i] == x) {  
        for (int j = i; j < tope - 1; j++) {  
            numerito[j] = numerito[j + 1];  
        }  
        tope--;  
        break;  
    }  
}
```

8 12 5 9 4



8 12 9 4

EJEMPLO PARA DECIR EN CLASE

"Primero busco X. Al encontrarlo, corro los elementos que están después una posición hacia la izquierda. Como el arreglo pierde un dato válido, el tope disminuye."

IDEA CLAVE

Desplazamiento a la izquierda

ESTRUCTURA

for + comparación + desplazamiento

RESULTADO

El elemento X desaparece y el arreglo queda compacto.



CONCEPTO CENTRAL

Busca un elemento específico X dentro del arreglo y, al encontrarlo, reemplaza su contenido por un nuevo valor denominado nuevoValor.

¿CÓMO FUNCIONA?

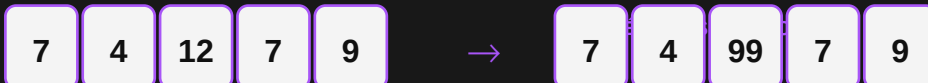
- Recorrer las posiciones activas del arreglo.
- Comparar numerito[i] con X.
- Si coinciden, asignar nuevoValor a esa posición.
- No se modifica el tamaño del arreglo: el tope permanece igual.

PASO A PASO PARA EXPONER

- 1 Recorrer las posiciones activas del arreglo.
- 2 Comparar numerito[i] con X.
- 3 Si coinciden, asignar nuevoValor a esa posición.
- 4 No se modifica el tamaño del arreglo: el tope permanece igual.

CÓDIGO / LÓGICA JAVA

```
for (int i = 0; i < tope; i++) {  
    if (numerito[i] == x) {  
        numerito[i] = nuevoValor;  
        break;  
    }  
}
```



EJEMPLO PARA DECIR EN CLASE

"Aquí no elimino ni agrego posiciones. Solo localizo el valor X y sustituyo ese dato por el nuevo valor. Por eso el tope no cambia."

IDEA CLAVE

Reemplazo directo

ESTRUCTURA

recorrido + if + asignación

RESULTADO

El valor seleccionado cambia sin alterar el tamaño.



CONCEPTO CENTRAL

Comprueba que la posición Y sea válida y que exista espacio disponible. Después desplaza los elementos hacia la derecha para abrir un espacio e inserta X en la posición indicada.

¿CÓMO FUNCIONA?

- Validar que Y esté dentro de las posiciones permitidas.
- Verificar que el arreglo tenga espacio.
- Desplazar desde el final hacia Y.
- Colocar X en `numerito[Y]`.
- Incrementar el tope.

PASO A PASO PARA EXPONER

- 1 Validar que Y esté dentro de las posiciones permitidas.
- 2 Verificar que el arreglo tenga espacio.
- 3 Desplazar desde el final hacia Y.
- 4 Colocar X en `numerito[Y]`.
- 5 Incrementar el tope.

CÓDIGO / LÓGICA JAVA

```
if (y >= 0 && y <= tope && tope < numerito.length) {  
    for (int i = tope; i > y; i--) {  
        numerito[i] = numerito[i - 1];  
    }  
    numerito[y] = x;  
    tope++;  
}
```

10 20 30 40



10 99 20 30 40

EJEMPLO PARA DECIR EN CLASE

"Para insertar no puedo sobrescribir el dato que ya existe. Primero muevo los elementos hacia la derecha, dejo un espacio libre y recién coloco X en la posición Y."

IDEA CLAVE

Abrir espacio desplazando

ESTRUCTURA

validación + for inverso + inserción

RESULTADO

X ocupa Y y los demás datos conservan su orden.

CONCEPTO CENTRAL

Recorre todos los elementos almacenados y acumula sus valores en una variable sumadora. El resultado representa la suma total del arreglo hasta tope - 1.

¿CÓMO FUNCIONA?

- Inicializar suma en cero.
- Recorrer las posiciones activas.
- Agregar cada elemento a suma.
- Retornar o mostrar el acumulado final.

PASO A PASO PARA EXPONER

- 1 Inicializar suma en cero.
- 2 Recorrer las posiciones activas.
- 3 Agregar cada elemento a suma.
- 4 Retornar o mostrar el acumulado final.

CÓDIGO / LÓGICA JAVA

```
int suma = 0;

for (int i = 0; i < tope; i++) {
    suma += numerito[i];
}

return suma;
```

5 8 2 10 4



suma = 29

RESULTADO

EJEMPLO PARA DECIR EN CLASE

"La variable suma comienza en cero y funciona como un acumulador. En cada vuelta agrego el valor actual y al terminar obtengo la suma de todos los elementos."

IDEA CLAVE

Acumulador

ESTRUCTURA

suma = 0 + recorrido

RESULTADO

Se obtiene la suma de los elementos válidos.

CONCEPTO CENTRAL

Recorre el arreglo para encontrar simultáneamente el número menor y el mayor, además de acumular la suma para calcular el promedio usando el valor de tope.

¿CÓMO FUNCIONA?

- Tomar el primer elemento como menor y mayor inicial.
- Comparar cada elemento restante.
- Actualizar menor y mayor cuando corresponda.
- Acumular la suma.
- Calcular promedio = suma / tope.

PASO A PASO PARA EXPONER

- 1 Tomar el primer elemento como menor y mayor inicial.
- 2 Comparar cada elemento restante.
- 3 Actualizar menor y mayor cuando corresponda.
- 4 Acumular la suma.
- 5 Calcular promedio = suma / tope.

CÓDIGO / LÓGICA JAVA

```
int menor = numerito[0];
int mayor = numerito[0];
int suma = 0;

for (int i = 0; i < tope; i++) {
    if (numerito[i] < menor) menor = numerito[i];
    if (numerito[i] > mayor) mayor = numerito[i];
    suma += numerito[i];
}

double promedio = (double) suma / tope;
```

8 3 15 6 10



Menor = 3 Mayor = 15 Promedio = 8.4

EJEMPLO PARA DECIR EN CLASE

"Uso el primer dato como referencia inicial. Mientras recorro el arreglo voy actualizando el menor y el mayor, y al mismo tiempo acumulo la suma para obtener el promedio."

IDEA CLAVE

Tres resultados en un recorrido

ESTRUCTURA

menor + mayor + suma

RESULTADO

Se obtienen mínimo, máximo y media aritmética.



SUMA DE IMPARES

ARREGLOS • JAVA • ALGORITMOS • SEMANA 02



CONCEPTO CENTRAL

Evalúa cada elemento mediante el operador residuo. Si el resultado de dividir entre 2 es diferente de cero, el número es impar y se agrega al acumulador.

¿CÓMO FUNCIONA?

- Inicializar sumaImpares en cero.
- Recorrer las posiciones activas.
- Evaluar `numerito[i] % 2 != 0`.
- Acumular solamente los valores impares.
- Mostrar la suma final.

PASO A PASO PARA EXPONER

- 1 Inicializar sumaImpares en cero.
- 2 Recorrer las posiciones activas.
- 3 Evaluar `numerito[i] % 2 != 0`.
- 4 Acumular solamente los valores impares.
- 5 Mostrar la suma final.

CÓDIGO / LÓGICA JAVA

```
int sumaImpares = 0;

for (int i = 0; i < tope; i++) {
    if (numerito[i] % 2 != 0) {
        sumaImpares += numerito[i];
    }
}

return sumaImpares;
```

4 7 2 9 6 5 → Impares: 7 Suma = 21

EJEMPLO PARA DECIR EN CLASE

“El operador % me permite saber si un número es impar. Si el residuo al dividir entre dos es diferente de cero, ese elemento entra en la suma.”

IDEA CLAVE

Filtro con `% 2 != 0`

ESTRUCTURA

recorrido + condición + acumulador

RESULTADO

Solo los números impares forman parte de la suma.



CONCEPTO CENTRAL

Guarda temporalmente el primer valor, desplaza los demás elementos una posición hacia la izquierda y coloca el valor guardado en la última posición activa.

¿CÓMO FUNCIONA?

- Guardar `numerito[0]` en una variable temporal.
- Desplazar `numerito[1]`, `numerito[2]`, etc. hacia la izquierda.
- Colocar temporal en `numerito[tope - 1]`.
- El tamaño lógico del arreglo no cambia.

PASO A PASO PARA EXPONER

- 1 Guardar `numerito[0]` en una variable temporal.
- 2 Desplazar `numerito[1]`, `numerito[2]`, etc. hacia la izquierda.
- 3 Colocar temporal en `numerito[tope - 1]`.
- 4 El tamaño lógico del arreglo no cambia.

CÓDIGO / LÓGICA JAVA

```
int primero = numerito[0];

for (int i = 0; i < tope - 1; i++) {
    numerito[i] = numerito[i + 1];
}

numerito[tope - 1] = primero;
```

10 20 30 40 50



20 30 40 50 10

EJEMPLO PARA DECIR EN CLASE

"Primero guardo el dato de la posición cero porque lo voy a mover. Después corro todos los demás hacia la izquierda y finalmente coloco el valor guardado al final."

IDEA CLAVE

Rotación hacia la izquierda

ESTRUCTURA

temporal + desplazamiento + asignación

RESULTADO

El primer elemento pasa a la última posición.

CONCEPTO CENTRAL

Busca el primer número par del arreglo. Cuando lo encuentra, guarda su valor, desplaza los elementos siguientes hacia la izquierda y coloca ese número par al final de los elementos válidos.

¿CÓMO FUNCIONA?

- Buscar desde el inicio hasta encontrar el primer par.
- Comprobar con `numero[i] % 2 == 0`.
- Guardar la posición y el valor del par.
- Desplazar los elementos posteriores una posición a la izquierda.
- Colocar el par en `tope - 1`.

PASO A PASO PARA EXPONER

1. Buscar desde el inicio hasta encontrar el primer par.
2. Comprobar con `numero[i] % 2 == 0`.
3. Guardar la posición y el valor del par.
4. Desplazar los elementos posteriores una posición a la izquierda.
5. Colocar el par en `tope - 1`.

CÓDIGO / LÓGICA JAVA

```
int posPar = -1;

for (int i = 0; i < tope; i++) {
    if (numero[i] % 2 == 0) {
        posPar = i;
        break;
    }
}

if (posPar != -1) {
    int par = numero[posPar];

    for (int i = posPar; i < tope - 1; i++) {
        numero[i] = numero[i + 1];
    }
    numero[tope - 1] = par;
}
```



EJEMPLO PARA DECIR EN CLASE

"Recorro desde el principio y me detengo en el primer par. Lo guardo, cierro el espacio desplazando los siguientes elementos y finalmente pongo ese par al final."

IDEA CLAVE

Primer par + desplazamiento

ESTRUCTURA

búsqueda + `% 2` + movimiento

RESULTADO

El primer número par termina al final del arreglo.